

```

import cv2 # Import OpenCV for image processing algorithms
import numpy as np # Import Numpy for mathematical matrix operations
import matplotlib.pyplot as plt # Import Matplotlib to visualize the output on screen

# =====
# STEP 1: CREATE A CLEAN SYNTHETIC INPUT PATTERN
# =====
# Generate a simple 120x120 grayscale checkerboard target
clean_img = np.zeros((120, 120), dtype=np.uint8)
clean_img[30:90, 30:90] = 180 # Solid light gray square core

# =====
# STEP 2: SIMULATE ACQUISITION PROCESS NOISE (Sensor Grain)
# =====
# Generate zero-mean Gaussian electronic noise arrays
noise = np.random.normal(0, 25, clean_img.shape).astype(np.int16)
noisy_img = np.clip(clean_img.astype(np.int16) + noise, 0, 255).astype(np.uint8)

# =====
# STEP 3: MINIMIZE NOISE WITH LOW-PASS MIGRATION FILTERS
# =====
# Use Gaussian Blur to smooth out high-frequency fluctuations
denoised_img = cv2.GaussianBlur(noisy_img, (5, 5), 0)

# =====
# STEP 4: VISUALIZE RESULTS SIDE-BY-SIDE (COMPACT SIZE)
# =====
fig, axes = plt.subplots(1, 3, figsize=(6, 2))

axes[0].imshow(clean_img, cmap='gray', vmin=0, vmax=255)
axes[0].set_title('Clean Original', fontsize=8)
axes[0].axis('off')

axes[1].imshow(noisy_img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title('Sensor Noise Added', fontsize=8)
axes[1].axis('off')

axes[2].imshow(denoised_img, cmap='gray', vmin=0, vmax=255)
axes[2].set_title('Denoised (Cleaned)', fontsize=8)
axes[2].axis('off')

plt.tight_layout()
plt.show()

```

